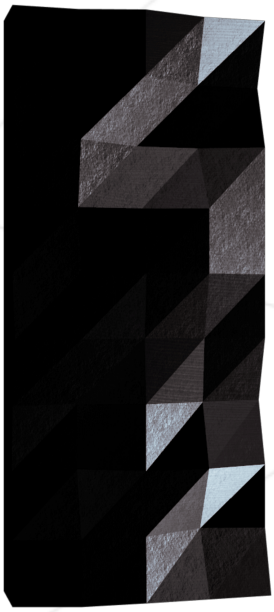

The Codex

Mastering Python's Import Mysteries

Summary: Discover the ancient art of Python imports through alchemical experiments. Master the four sacred mysteries: package initialization, import pathways, absolute vs relative access, and breaking circular dependencies.

Version: 2.0

Contents

I AI Instructions **2**

II Introduction **4**

III General Instructions **5**

IV Mandatory part **7**

IV.1 Part I: The Alembic 7

IV.2 Part II: Distillation 10

IV.3 Part III: The Great Transmutation 12

IV.4 Part IV: Avoid the Explosion 14

V Submission Instructions **16**

Chapter I

AI Instructions

Context

During your learning journey, AI can assist with many different tasks. Take the time to explore the various capabilities of AI tools and how they can support your work. However, always approach them with caution and critically assess the results. Whether it's code, documentation, ideas, or technical explanations, you can never be completely sure that your question was well-formed or that the generated content is accurate. Your peers are a valuable resource to help you avoid mistakes and blind spots.

Main message

- Use AI to reduce repetitive or tedious tasks.
- Develop prompting skills — both coding and non-coding — that will benefit your future career.
- Learn how AI systems work to better anticipate and avoid common risks, biases, and ethical issues.
- Continue building both technical and power skills by working with your peers.
- Only use AI-generated content that you fully understand and can take responsibility for.

Learner rules:

- You should take the time to explore AI tools and understand how they work, so you can use them ethically and reduce potential biases.
- You should reflect on your problem before prompting — this helps you write clearer, more detailed, and more relevant prompts using accurate vocabulary.
- You should develop the habit of systematically checking, reviewing, questioning, and testing anything generated by AI.
- You should always seek peer review — don't rely solely on your own validation.

Phase outcomes:

- Develop both general-purpose and domain-specific prompting skills.
- Boost your productivity with effective use of AI tools.
- Continue strengthening computational thinking, problem-solving, adaptability, and collaboration.

Comments and examples:

- You'll regularly encounter situations — exams, evaluations, and more — where you must demonstrate real understanding. Be prepared, keep building both your technical and interpersonal skills.
- Explaining your reasoning and debating with peers often reveals gaps in your understanding. Make peer learning a priority.
- AI tools often lack your specific context and tend to provide generic responses. Your peers, who share your environment, can offer more relevant and accurate insights.
- Where AI tends to generate the most likely answer, your peers can provide alternative perspectives and valuable nuance. Rely on them as a quality checkpoint.

✓ Good practice:

> I ask AI: "How do I test a sorting function?" It gives me a few ideas. I try them out and review the results with a peer. We refine the approach together.

✗ Bad practice:

> I ask AI to write a whole function, copy-paste it into my project. During peer-evaluation, I can't explain what it does or why. I lose credibility — and I fail my project.

✓ Good practice:

> I use AI to help design a parser. Then I walk through the logic with a peer. We catch two bugs and rewrite it together — better, cleaner, and fully understood.

✗ Bad practice:

> I let Copilot generate my code for a key part of my project. It compiles, but I can't explain how it handles pipes. During the evaluation, I fail to justify and I fail my project.

Chapter II

Introduction

Welcome to the Alchemist's Laboratory, young apprentice!

In medieval times, alchemists sought to transform base metals into gold. Today, as a Python alchemist, you'll learn to transform scattered code into organized, reusable magical formulas. Just as ancient alchemists had their grimoires (spell books) organized into chapters and sections, Python code must be organized into modules and packages.

But beware! The path of the alchemist is fraught with mysteries:

- Using `__init__.py`, the gateway that transforms a simple folder into a magical package
- Calling the code of a distant module from a distant module
- Managing absolute vs relative imports
- Avoiding circular dependencies

In this codex, you'll build your own alchemical laboratory: a complete Python package that demonstrates these four sacred mysteries.

> Focus on understanding **how imports work**, not just making them work.
> A true alchemist understands every step of the transmutation process!

Chapter III

General Instructions

- Your project must be written in **Python 3.10 or later**.
- Your project must adhere to the **flake8** coding standard.
- All code must include comprehensive type annotations. Check this using `mypy` .
- All standard classes and collections are authorized, along with their methods (`int`, `str`, `list`, `dict`, etc.).
- All built-in functions are authorized, but `eval()` and `exec()`.
- Only imports of the modules and files you will create in this project are allowed.
- It is forbidden to modify `sys.path` .

This project is split into four parts that will build a unique structure of files and folders. At the end of the project, your tree should look like:

```
.
|-- alchemy
|   |-- __init__.py
|   |-- elements.py
|   |-- grimoire
|   |   |-- __init__.py
|   |   |-- dark_spellbook.py
|   |   |-- dark_validator.py
|   |   |-- light_spellbook.py
|   |   `-- light_validator.py
|   |-- potions.py
|   `-- transmutation
|       |-- __init__.py
|       `-- recipes.py
|-- elements.py
|-- ft_alembic_0.py
|-- ft_alembic_1.py
|-- ft_alembic_2.py
|-- ft_alembic_3.py
|-- ft_alembic_4.py
|-- ft_alembic_5.py
|-- ft_distillation_0.py
|-- ft_distillation_1.py
|-- ft_kaboom_0.py
|-- ft_kaboom_1.py
|-- ft_transmutation_0.py
|-- ft_transmutation_1.py
`-- ft_transmutation_2.py
```

All functions should be simple and return strings. Focus on import concepts, not complex logic.

Chapter IV

Mandatory part

IV.1 Part I: The Alembic

Every alchemist starts with simple ingredients in their alembic! But sometimes, some ingredients are close, others are not. This first part is about using `import` mechanisms to access local files and modules.

First, we will create the components of our laboratory architecture (and then the testing suite).

elements.py will contain:

- A function `create_fire()` that just returns the string "Fire element created".
- A function `create_water()` that just returns the string "Water element created".

alchemy/elements.py will contain:

- A function `create_earth()` that just returns the string "Earth element created".
- A function `create_air()` that just returns the string "Air element created".

alchemy/__init__.py will contain:

- Anything useful to partially import the `alchemy` module. See `ft_alembic_4.py` .

In order to carefully test our laboratory architecture, create the following six scripts:

- `ft_alembic_0.py` will use the `import ...` structure to access `elements.py` directly and then create fire.
- `ft_alembic_1.py` will use the `from ... import ...` structure to access `elements.py` directly and then create water.
- `ft_alembic_2.py` will use the `import ...` structure to access `alchemy/elements.py` directly and then create earth.

- `ft_alembic_3.py` will use the `from ... import ...` structure to access `alchemy/elements.py` directly and then create air.
- `ft_alembic_4.py` will use `import alchemy` to access the alchemy module and then create air. The `create_earth()` function will not be exposed through the module interface and raise an exception when called (you can catch the exception or not, this is only for pedagogical purposes). A `mypy` error will also raise, again, on purpose.
- `ft_alembic_5.py` will use the `from alchemy import ...` structure to access the alchemy module and then create air.

> You may face flake8 / mypy errors over your __init__.py file. Follow usual conventions in this situation.

Example:

```
$> python3 ft_alembic_0.py
=== Alembic 0 ===
Using: 'import ...' structure to access elements.py
Testing create_fire: Fire element created

$> python3 ft_alembic_1.py
=== Alembic 1 ===
Using: 'from ... import ...' structure to access elements.py
Testing create_water: Water element created

$> python3 ft_alembic_2.py
=== Alembic 2 ===
Accessing alchemy/elements.py using 'import ...' structure
Testing create_earth: Earth element created

$> python3 ft_alembic_3.py
=== Alembic 3 ===
Accessing alchemy/elements.py using 'from ... import ...' structure
Testing create_air: Air element created

$> python3 ft_alembic_4.py
=== Alembic 4 ===
Accessing the alchemy module using 'import alchemy'
Testing create_air: Air element created
Now show that not all functions can be reached
This will raise an exception!
Testing the hidden create_earth: Traceback (most recent call last):
  File "/tmp/Python/module-06/ft_alembic_4.py", line 12, in <module>
    print(f"{alchemy.create_earth()}")
            ^^^^^^^^^^^^^^^^^^^^^
AttributeError: module 'alchemy' has no attribute 'create_earth'. Did you mean: 'create_air'?

$> python3 ft_alembic_5.py
=== Alembic 5 ===
Accessing the alchemy module using 'from alchemy import ...'
Testing create_air: Air element created
```

IV.2 Part II: Distillation

Master the art of nested import and summon distant codes from distant files.

Let's improve our laboratory architecture and add a new component:

alchemy/potions.py will contain:

- A function `healing_potion()` that returns the string "Healing potion brewed with '[created earth element]' and '[created air element]".
- A function `strength_potion()` that returns the string "Strength potion brewed with '[created fire element]' and '[created water element]".
- Anything else useful to access the four fundamental elements.

In order to check these new functionalities, create the following two scripts:

- `ft_distillation_0.py` will use the `from ... import ...` structure to access `alchemy/potions.py` directly and then brew both healing and strength potions.
- `ft_distillation_1.py` will use the `import alchemy` structure to access the alchemy module, then brew the original strength potion and the special `heal()` potion, which is a package *alias* of the healing potion. You will need to improve your `__init__.py` .

Example:

```
$> python3 ft_distillation_0.py
=== Distillation 0 ===
Direct access to alchemy/potions.py
Testing strength_potion: Strength potion brewed with 'Fire element created' and 'Water element created'
Testing healing_potion: Healing potion brewed with 'Earth element created' and 'Air element created'

$> python3 ft_distillation_1.py
=== Distillation 1 ===
Using: 'import alchemy' structure to access potions
Testing strength_potion: Strength potion brewed with 'Fire element created' and 'Water element created'
Testing heal alias: Healing potion brewed with 'Earth element created' and 'Air element created'
```

IV.3 Part III: The Great Transmutation

Understand the ancient debate between absolute and relative imports: two different paths to reach the same magical formula.

Let's improve once again our laboratory architecture and add a new component:

alchemy/transmutation/recipes.py will contain:

- A function `lead_to_gold()` that returns the string "Recipe transmuting Lead to Gold: brew '[created air]' and '[created strength potion]' mixed with '[created fire]'".
- At least one absolute import and one relative import.

Create three testing scripts to verify everything imports successfully:

- `ft_transmutation_0.py` will use the `import ...` structure to access `alchemy/transmutation/recipes.py` directly and then achieve transformation of lead into gold.
- `ft_transmutation_1.py` will use the `import ...` structure to import the module `transmutation` directly and then achieve transformation of lead into gold.
- `ft_transmutation_2.py` will use the `import ...` structure to import the module `alchemy` directly and then achieve transformation of lead into gold.

You may need to create and/or update a few extra files to complete these scripts.

Example:

```
$> python3 ft_transmutation_0.py
=== Transmutation 0 ===
Using file alchemy/transmutation/recipes.py directly
Testing lead to gold: Recipe transmuting Lead to Gold: brew 'Air element created' and 'Strength potion
     brewed with 'Fire element created' and 'Water element created'' mixed with 'Fire element created'

$> python3 ft_transmutation_1.py
=== Transmutation 1 ===
Import transmutation module directly
Testing lead to gold: Recipe transmuting Lead to Gold: brew 'Air element created' and 'Strength potion
     brewed with 'Fire element created' and 'Water element created'' mixed with 'Fire element created'

$> python3 ft_transmutation_2.py
=== Transmutation 2 ===
Import alchemy module only
Testing lead to gold: Recipe transmuting Lead to Gold: brew 'Air element created' and 'Strength potion
     brewed with 'Fire element created' and 'Water element created'' mixed with 'Fire element created'
```

When should an alchemist use absolute pathways vs relative pathways?

IV.4 Part IV: Avoid the Explosion

Learn to identify and break the explosive circular dependency curse, which occurs when modules try to summon each other in an endless loop, threatening to destroy your laboratory!

Finally, we will add these last components in our laboratory architecture:

alchemy/grimoire/light_spellbook.py will contain:

- A function `light_spell_allowed_ingredients()` that returns a list of allowed ingredients for light magic, let's say "earth", "air", "fire", "water".
- A function `light_spell_record(spell_name: str, ingredients: str)` that returns a string that indicates whether the spell is recorded or rejected. The decision comes from the validation function described below.

alchemy/grimoire/light_validator.py will contain:

- A function `validate_ingredients(ingredients: str)` that returns a string with the ingredients and the "VALID" or "INVALID" keyword. The ingredients are valid if they include at least one of the allowed ingredients from the spellbook (case insensitive).

Now duplicate both files to obtain `dark_spellbook.py` and `dark_validator.py`, and change the function names as well. Let's say that dark magic uses the following ingredients: "bats", "frogs", "arsenic", and "eyeball".

Time to test a few spells! Create two scripts to demonstrate that light magic avoids circular dependencies and laboratory explosions, and that dark magic is dangerous and explodes because of circular dependencies:

- `ft_kaboom_0.py` will access the `grimoire` module directly and then record a light spell, flawlessly. There are multiple ways to avoid circular dependencies; you pick one. Be prepared to explain the different approaches during the evaluation.
- `ft_kaboom_1.py` will secretly access the `dark_spellbook.py` directly and then try to record a dark spell. This must fail and raise an exception (you can choose to catch it or not; it's only for pedagogical purposes), indicating that your alchemist laboratory has just exploded.

Example:

```
$> python3 ft_kaboom_0.py
=== Kaboom 0 ===
Using grimoire module directly
Testing record light spell: Spell recorded: Fantasy (Earth, wind and fire - VALID)

$> python3 ft_kaboom_1.py
=== Kaboom 1 ===
Access to alchemy/grimoire/dark_spellbook.py directly
Test import now - THIS WILL RAISE AN UNCAUGHT EXCEPTION
Traceback (most recent call last):
  File "/tmp/Python/module-06/ft_kaboom_1.py", line 7, in <module>
    from alchemy.grimoire.dark_spellbook import dark_spell_record
  File "/tmp/Python/module-06/alchemy/grimoire/dark_spellbook.py", line 2, in <module>
    from .dark_validator import validate_ingredients
  File "/tmp/Python/module-06/alchemy/grimoire/dark_validator.py", line 2, in <module>
    from .dark_spellbook import dark_spell_allowed_ingredients
ImportError: cannot import name 'dark_spell_allowed_ingredients' from partially initialized module '
     alchemy.grimoire.dark_spellbook' (most likely due to a circular import) (/tmp/Python/module-06/
     alchemy/grimoire/dark_spellbook.py)
```

Chapter V

Submission Instructions

Turn in your assignment in your `Git` repository, as usual. Only the work inside your repository will be evaluated during the defense.

Place the expected file tree at the root of your Git repository.

> During evaluation, you may be asked to explain import mechanisms, demonstrate different import styles, or modify your alchemical laboratory. Make sure you understand the four sacred mysteries, not just their implementation.